

object) whenever we want, not just when a program starts. In this program, we create a new `Account` object whenever a user decides to open an account. Our code can use that object by calling its methods. We can also delete an object at any time, in this case, when a user chooses to close their account. This is an example of how an object (like an `Account` object) has a life span, from whenever it is created to whenever it is deleted.

Better Error Handling with Exceptions

So far in our `Account` class, if a method detects an error (for example, if the user deposits a negative amount, enters an incorrect password, withdraws a negative amount, and so on) our placeholder solution is to return `None` as a signal that something went wrong. In this section, we'll discuss a better way of handling errors by using `try/except` blocks and raising exceptions.

try and except

When a runtime error or abnormal condition occurs in a function or method from the Python Standard Library, that function or method signals the error by raising an exception (sometimes referred to as *throwing* or *generating* an exception). We can detect and react to exceptions using the `try/except` construct. Here is the general form:

```
try:
    # some code that may cause an error (raise an exception)
except <some exception name>: # if an exception happens
    # some code to handle the exception
```

If the code inside the `try` block works correctly and does not generate an exception, the `except` clause is skipped, and execution continues after the `except` block. However, if the code in the `try` block results in an exception, control is passed to the `except` statement. If the exception matches the exception (or one of multiple exceptions) listed in the `except` statement, control is transferred to the code of the `except` clause. This is often referred to as *catching* the exception. That indented block typically contains code to report and/or recover from the error.